Testing de caja negra CLASES GRABADAS

Que es el testing? El testing es un proceso destructivo cuyo objetivo es encontrar defectos, es decir poder asumir que alguna porción de software va a tener defectos y asi encontrarlos, para posteriormente resolverlos y contribuir a la calidad de un producto de software.

Cuando testing es suficiente? Depende de diferentes criterios, pero no se puede hacer testing exhaustivo. Se corta cuando la cantidad de defectos encontrados llegue a cierto umbral, cuando se llegue a cierto presupuesto

Ágil por su parte se inclina por el lado de la automatización de las pruebas para economizar los recursos y poder repetir las pruebas, con las pruebas de regresión de manera que la solución de un defecto no implique la introducción de otros nuevos defectos

Error vs Defecto: ambas son fallas en un producto del software, todo depende del momento en el que se encuentran y teniendo en cuenta cuando se producen

Error: cuando encontramos una falla durante la misma etapa que la origino, por ejemplo estamos en la etapa de implementacion y ahí mismo encontramos una falla. tiene un menor costo.

Defecto: cuando encontramos una falla durante una etapa posterior a la que la origino, por ejemplo estamos en testing y encontramos una falla que se origino en el desarrollo. tiene un mayor costo.

El testing unitario busca encontrar defectos pero a su vez puede encontrar errores ya que se realiza durante la implementacion.

Proceso del testing
Etapas:

1. Planificación y control:
2. Analisis y diseño: aqui aparecen los casos de prueba los cuales son los artefactos por excelencia del testing como disciplina y van a servir como guia para la ejecución de las pruebas para luego tomar una retroalimentación en base a los resultados.
3. Ejecución
4. Evaluación y reportes

Niveles de testing

1. Testing unitario: se desarrolla durante la implementacion, busca de proveer de un mecanismo de verificación de la comprension de la funcionalidad que estoy implementando. Escribo los CP unitarios que me permiten garantizar que la funcionalidad que estoy implementando cumple con el requerimiento dado. no busca encontrar defectos, sino errores. lo lleva adelante el desarrollador
2. Testing de integración: buscamos encontrar defectos porque esta orientado a probar las interfaces o componentes de alto y bajo nivel. Se busca encontrar defectos entre la union de diferentes componentes a través de sus interfaces. Lo lleva adelante el tester. las distintas estrategias que existen son:
    a. bottom up: desde lo mas chico a lo mas grande

    b. top down: al reves de bottom up
    c. sandwich: intercalando bottom up y top down
3. testing de sistema: probamos una funcionalidad en su totalidad. el tester ejecuta cierto camino o escenario con ciertas características para poder encontrar si el resultado obtenido de la ejecución de esa funcionalidad coincide con el resultado esperado para encontrar defectos que hayan surgido de la implementacion. Lo lleva adelante el tester.
4. Testing de aceptacion: Lo lleva adelante el cliente/usuario o PO el cual nos va a dar una respuesta a asi sus requerimientos estan realmente bien implementados con su aceptación. Se valida que lo que se entrega esta realmente correcto y alineado con los requerimientos. Se realiza en un ambiente producto ya sea con una prueba alpha o beta.

Caso de prueba: artefacto por excelencia del testing , es la guia. contiene un conjuntos de condiciones o pasos que se ceben ejecutar para poder garantizar que el resultado esperado sea o no igual al resultado obtenido y de esa forma encontrar defectos. Diseñar la mínima cantidad de CP para maximizar la cantidad de defectos encontrados.
Es una receta, es un conjunto de pasos ordenados que yo debo seguir para ejecutar la funcionalidad con una especificación de los valores de cuales son los valores a ingresar. debe especificar valores que son miembros de alguna clase de equivalencia de esa condición externa.

Por lo que teniendo en cuenta las infinitas posibilidades y valores que tendremos para probar en testing buscamos estrategia saque buscan maximizar la cantidad de defectos encontrados minimizando el esfuerzo para hacerlo. Por lo que para el diseño de casos de prueba surgen dos estrategias:

1. Caja blanca: Podemos ver el detalle de la implementacion realizada, por lo que podemos ver el codigo. y diseñar los CP para poder garantizar la cobertura. conozco la estructura interna.
2. caja negra: no conozco la estructura interna de la implementacion que solo se va a analizar en entradas y salidas. voy a identificar cuales son las entradas que una funcionalidad va a tener , voy a elegir los valores con los cuales voy a ejecutar esa funcionalidad y finalmente voy a comparar las salidas obtenidas con las esperadas.

Lo ideal es aplicar ambas estrategias para distintas cosas

Dentro de cada estrategia tenemos distintos métodos para implementar cada estrategia en particular, se hace para maximizar la cantidad de defectos encontrados. los métodos sirven para diseñar los casos de prueba.

Dentro de caja negra tenemos:

1. Basado en especificaciones: partición de equivalencias y el análisis de valores limites
    a. Partición de equivalencias : analiza primero cuales son las distintas condiciones externas que van a estar involucradas en una funcionalidad, son tanto las entradas como las salidas posibles. cualquier condición que va a guiar la ejecución de la funcionalidad , ya sea entrada o salida. Para cada uno voy a analizar los subconjuntos de valores posibles que pueden tomar

cada uno de esas condiciones externas que van a producir un resultado equivalente. Es un subconjunto de valores que puede tomar una condición externa para el cual si yo tomo cualquier valor de ese subconjunto el resultado en la ejecución de la funcionalidad es equivalente.

i. Rango de valores continuos
ii. Valores Discretos
iii. Selección simple
iv. Selección múltiple

b. Análisis de valores límites: Es una variante de la partición de equivalencias en la que en vez de seleccionar cualquier elemento representativo de una clase de equivalencia , se seleccionan los bordes de una clase. esto ya que la mayoría de los defectos se encuentran en los extremos de las clases de equivalencia.

2. Basado en la experiencia: adivinanza de defectos y testing exploratorio

Prioridad: se basa en que si tenemos que seleccionar Cp para ejecutar antes que otros vamos a priorizar aquellos que nos permitan encontrar mayor cantidad de defectos y que garanticen la salud de la funcionalidad. los de camino feliz son de prioridad alta. Los de prioridad baja son aquellas que están relacionadas con validaciones, valores no ingresados, con formatos distintos. prioridad media todos los que están en el medio.

Nombre CP: el nombre debe ser representativo en cuanto al escenario que se ejecutará.

Precondiciones: conjunto de valores o características que tiene que tener el contexto para que yo pueda llevar adelante este caso de prueba particular

Pasos: conjunto de operaciones ordenadas y enumeradas. Por ejemplo el ROL ingresa a la opción “...” . Son claros y sin ambigüedades que deben ejecutar nuestro tester sobre la funcionalidad para conseguir un resultado esperado

Resultado esperado: todos los resultados que se esperan por parte del sistema en ese caso de prueba especifico.